

📖 AGENTIC AI — MASTER STUDY GUIDE

LangChain · LangGraph · RAG · Memory · Multi-Agent Systems · Production
Deployment | 2026 Edition

📖 *This master guide covers the complete Agentic AI curriculum — from LangChain basics to production-grade multi-agent systems. Every concept is explained in plain, easy-to-understand language with code examples, tables, and practical tips.*

TABLE OF CONTENTS

- MODULE 1:** LangChain Foundations — Ecosystem, Setup & Core Concepts
- MODULE 2:** Chain Patterns — Sequential, Parallel, Branching & Debugging
- MODULE 3:** Document Loading, Chunking & Embeddings
- MODULE 4:** RAG — Retrieval-Augmented Generation (Basic → Advanced)
- MODULE 5:** Memory — Conversational Context & Persistence
- MODULE 6:** Project — AI Research Assistant (End-to-End RAG System)
- MODULE 7:** LangGraph — Stateful Agents & Graph Workflows
- MODULE 8:** LangGraph Multi-Agent Systems
- MODULE 9:** Production Deployment — Security, Testing, Cost & Monitoring

MODULE 1 — LangChain Foundations

LangChain is a framework for building AI-powered applications using Large Language Models (LLMs). Think of it as a toolbox that lets you connect LLMs with data, tools, memory, and logic — all in clean Python code.

1.1 Why LangChain & LangGraph in 2026?

LangChain has grown from a simple chain library into a full AI infrastructure platform:

- Vendor Agnostic — Swap OpenAI ↔ Anthropic ↔ Ollama with minimal code changes
- LangSmith — Built-in tracing, monitoring, and evaluation for LLM apps
- LangGraph — Durable, resumable workflows with persistent state (the real innovation)
- Used in production by Uber, LinkedIn, Klarna and many more
- Company valuation: \$1.25 Billion | AI Agent Framework market growing ~50%/year

1.2 The LangChain Ecosystem — Package Map

Package	What It Does	When to Use It
langchain-core	Core abstractions, LCEL, Runnable	Always — it's the foundation
langchain	Agents, chains, high-level APIs	Building chains, RAG, quick prototypes
langgraph	Stateful agent orchestration	Stateful agents, loops, multi-agent, production
langsmith	Tracing, evaluation, monitoring	Debugging, monitoring, evaluating LLMs
langserve	Deploy workflows as REST APIs	Exposing agents/chains to client apps
langchain-openai	OpenAI integration	Using GPT models
langchain-anthropic	Anthropic integration	Using Claude models

1.3 Development Environment Setup

```
# Install UV and create project
curl -LsSf https://astral.sh/uv/install.sh | sh
uv init && uv venv && source .venv/bin/activate

# Install packages
uv add langchain langchain-core langgraph langchain-openai langchain-anthropic python-dotenv

# .env file
OPENAI_API_KEY=sk-...
ANTHROPIC_API_KEY=sk-ant-...
LANGSMITH_API_KEY=ls-...
```

1.4 Core Concept: LCEL & Runnable

LCEL (LangChain Expression Language) uses the pipe operator | to chain components. Every component is a Runnable — it supports .invoke(), .batch(), and .stream().

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_openai import ChatOpenAI

prompt = ChatPromptTemplate.from_template('Answer in one sentence: {question}')
model = ChatOpenAI(model='gpt-4o-mini', temperature=0.7)
parser = StrOutputParser()
```

```
chain = prompt | model | parser          # Build the chain
result = chain.invoke({'question': 'What is LangChain?'})
```

Method	What It Does	Example Use Case
<code>.invoke(input)</code>	Single input → single output	One question, one answer
<code>.batch([inputs])</code>	Multiple inputs processed in parallel	Translate 5 sentences at once
<code>.stream(input)</code>	Token-by-token streaming output	ChatGPT-like streaming UI

1.5 Message Types

Message Type	Role	When to Use
SystemMessage	system	Set AI behaviour, persona, instructions
HumanMessage	user	User's question or input
AIMessage	assistant	Previous AI response (multi-turn chat)
ToolMessage	tool	Results from tools, APIs, databases

1.6 Output Parsers — Getting Structured Data

Parser	Output Type	Best For
StrOutputParser	str	Simple text, chat interfaces
JsonOutputParser	dict	Need a Python dict from LLM output
<code>with_structured_output()</code>	Pydantic model	✓Modern recommended — typed, validated, one line

📌 Use `with_structured_output(YourPydanticModel)` — it's the cleanest, most reliable way to get structured data from an LLM. One line replaces 5 manual steps.

1.7 Cost Optimisation

Strategy	How	Savings
Choose the right model	Use gpt-4o-mini for simple tasks (17× cheaper than gpt-4o)	Up to 17×
Limit output tokens	Set <code>max_tokens</code> in model config	Significant (output costs 4-5× more)
Caching	Cache identical requests (in-memory, Redis, SQL)	Up to 100% for repeated queries

MODULE 2 — Chain Patterns

LangChain provides four core patterns for assembling workflows. Every component is a Runnable, so they all compose freely with the `|` operator.

2.1 Sequential Chain — Linear Step-by-Step

The simplest pattern: data flows left → right. Output of step N becomes input of step N+1.

```
chain = prompt | model | parser
result = chain.invoke({'text': 'LangChain is a framework for building LLM apps.'})
```

2.2 Parallel Chain — Run Multiple Chains at Once

`RunnableParallel` fans out one input to multiple chains that run concurrently, then collects results into a dictionary.

```
from langchain_core.runnables import RunnableParallel

analysis_chain = RunnableParallel(
    summary    = summarize_prompt | model | parser,
    keywords   = keywords_prompt  | model | parser,
    sentiment  = sentiment_prompt | model | parser,
)
results = analysis_chain.invoke({'text': text})
print(results['summary'], results['keywords'], results['sentiment'])
```

🔗 All three chains run in parallel — not one after another. This cuts total latency significantly when you need multiple analyses of the same input.

2.3 Pass-Through Chain — Essential for RAG

`RunnablePassthrough` is an identity function — it passes input through unchanged. In RAG systems, the retriever transforms your question into documents, losing the original question. `RunnablePassthrough` preserves it so the prompt gets both context AND the original question.

2.4 Conditional Branching — Smart Routing

`RunnableBranch` routes inputs to different chains based on a condition (which can itself be an LLM call).

```
branch = RunnableBranch(
    (is_code_question, code_prompt | model | parser),    # if code question
    (general_prompt | model | parser),                  # default
)
answer = branch.invoke({'input': 'How do I write a for loop in Python?'})
```

🔗 🔄 Branching uses 2 LLM calls per request: (1) the classifier to decide the route, (2) the selected chain to answer. Factor this into cost planning.

2.5 Debugging Chains — 3 Methods

Method	How	Best For
Schema Inspection	<code>chain.input_schema.model_json_schema()</code>	Verify input/output types before integration
<code>with_config()</code> Tracing	<code>chain.with_config({'run_name': 'my_chain'}).invoke(...)</code>	Named runs, LangSmith integration
<code>RunnableLambda</code> Tap	Insert lambda between steps to print intermediate data	Inspect live data at each step

MODULE 3 — Document Loading, Chunking & Embeddings

Before an LLM can answer questions about your documents, you need to: load them → split into chunks → convert to vectors → store in a vector database. This is the foundation of every RAG system.

3.1 Document Loaders

Loader	Use Case	Best For
TextLoader	Plain .txt files	Fast, simple
PyPDFLoader	PDF files	Simple PDFs, quick prototyping
PyMuPDFLoader	High-volume PDFs	Production use — fastest with rich metadata
WebBaseLoader	Web pages via URL	Scraping HTML content
DirectoryLoader	Entire folders of files	Batch loading with glob patterns
UnstructuredPDFLoader	Complex PDFs with tables/layouts	When layout matters

3.2 Why Chunking Matters — The Goldilocks Problem

Chunk Size	Effect on Retrieval	Effect on Context
Too Small (<200 chars)	✓Very precise	✗Loses context — incomplete thoughts
Just Right (500–1000)	✓Balanced precision	✓Enough context per chunk
Too Large (>2000 chars)	✗Noisy, imprecise	✓Rich context but hard to retrieve

📌 *Overlap is cheap insurance. Without overlap, important info split across chunk boundaries gets missed. Use 15–20% overlap (e.g. `chunk_size=500, chunk_overlap=100`).*

3.3 Text Splitters

Splitter	Best For	Needs Embeddings?
RecursiveCharacterTextSplitter	General text — most use cases	No
MarkdownHeaderTextSplitter	Docs, READMEs, wikis	No
from_language() Code Splitter	Source code — keeps functions intact	No
SemanticChunker	Mixed-topic documents	Yes (API calls)

3.4 Vectors & Embeddings — How AI Understands Meaning

An embedding model converts text into a list of numbers (a vector) that captures its meaning. Similar texts produce similar vectors. This is how AI knows 'dog' and 'puppy' are related, but 'dog' and 'car' are not.

Embedding Model	Dimensions	Cost	Best For
text-embedding-3-small	1536	Very cheap	✓General use — recommended
text-embedding-3-large	3072	Higher	High-accuracy production
HuggingFace (local)	384	Free	Development, privacy-sensitive
Ollama (local)	Varies	Free	Local testing

3.5 ChromaDB Vector Store

```
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(model='text-embedding-3-small')
vector_store = Chroma.from_documents(docs, embeddings, persist_directory='./chroma_db')

# Search
results = vector_store.similarity_search('What is LangChain?', k=3)

# Convert to retriever (plugs into chains)
retriever = vector_store.as_retriever(search_type='similarity', search_kwargs={'k': 3})
```


MODULE 4 — RAG (Retrieval-Augmented Generation)

RAG grounds LLM responses in your actual documents, dramatically reducing hallucination. Instead of relying on training data, RAG fetches relevant documents at query time and injects them as context.

4.1 Why RAG?

Problem with Raw LLMs	How RAG Solves It
Knowledge cutoff date	Feed new documents anytime — always up-to-date
No access to private data	Index your own documents into a vector store
Hallucination risk	Ground responses in real retrieved sources
No transparency	You can see exactly which documents were used

4.2 RAG — Three Phases

Phase 1 — Indexing (one-time setup):

Load documents → Chunk with text splitter → Embed with embedding model → Store in vector DB

Phase 2 — Retrieval (per query):

Embed the user's question → Similarity search in vector DB → Return top-K relevant chunks

Phase 3 — Generation:

Combine question + retrieved chunks in a prompt → Send to LLM → Get grounded answer

4.3 Basic RAG Chain

```
from langchain_core.runnables import RunnablePassthrough

def format_docs(docs):
    return '\n\n'.join(doc.page_content for doc in docs)

rag_chain = (
    {'context': retriever | format_docs, 'question': RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)
result = rag_chain.invoke('What is LangChain?')
```

📌 *Anti-Hallucination: Always include in your prompt: 'Answer based ONLY on the following context. If the answer is not in the context, say I don't know.' This prevents the LLM from making up answers.*

4.4 Advanced RAG Strategies

Strategy	Problem It Solves	How It Works
Multi-Query Retriever	Single query misses relevant docs	LLM generates 3-4 query variations; results merged & deduplicated
Contextual Compression	Full chunks contain irrelevant noise	LLM extracts only relevant parts from each chunk (85% token reduction)
Hybrid Search (BM25+Semantic)	Semantic search misses exact keywords	Combines keyword matching (40%) + semantic similarity (60%)
Parent Doc Retriever	Precision vs context tradeoff	Search small child chunks; return large parent chunks for full context
RAG with Sources	No citation / hard to verify	Tag each chunk with source; LLM cites sources in answer
RAG with Fallback	LLM hallucinates out-of-scope answers	Prompt instructs: 'Say I don't know if not in context'

MODULE 5 — Memory (Conversational Context)

Without memory, every LLM call is stateless — the model forgets everything after each response. Memory lets the AI remember names, preferences, and prior context across a conversation.

5.1 The Modern Memory Approach

Use `RunnableWithMessageHistory` — it wraps any chain and automatically manages conversation history per session. Each user gets their own isolated memory via a unique `session_id`.

```
from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are a helpful assistant.'),
    MessagesPlaceholder(variable_name='history'), # memory injected here
    ('human', '{input}')]

chain = prompt | llm | StrOutputParser()
chat = RunnableWithMessageHistory(chain, get_session_history,
    input_messages_key='input', history_messages_key='history')

config = {'configurable': {'session_id': 'user_001'}}
response = chat.invoke({'input': 'My name is Paolo.'}, config=config)
```

5.2 Memory Strategies Comparison

Strategy	Token Usage	Best For	Trade-off
Full Buffer	Grows linearly	Short conversations	Can overflow context window
Sliding Window	Fixed (last K)	Long conversations, cost control	Loses older context permanently
Summary Memory	Grows slowly	Long conversations with important facts	Slight info loss in compression
SQLite Persistence	Persistent	Production chatbots	Requires DB setup

5.3 Message Trimming

```
from langchain_core.messages import trim_messages

trimmed = trim_messages(
    messages,
    max_tokens=200,
    strategy='last', # keep most recent messages
    token_counter=llm,
    include_system=True, # always keep system prompt
    allow_partial=False
)
```

5.4 SQLite Persistent Memory

```
from langchain_community.chat_message_histories import SQLChatMessageHistory

def get_session_history(session_id: str):
    return SQLChatMessageHistory(
        session_id=session_id,
```

```
        connection_string='sqlite:///chat_history.db'  
    )
```

📌 *Production Tips: Use a unique session_id per user (e.g. UUID). Combine SQLite persistence with message trimming to prevent unbounded growth. For multi-server deployments, use PostgreSQL or Redis instead of SQLite.*

MODULE 6 — Project: AI Research Assistant

This end-to-end project combines every LangChain concept into one production-ready system. It accepts research queries, retrieves relevant document chunks, and returns grounded, structured answers with sources and confidence scores.

6.1 System Architecture

User Query → Conversation Memory → Multi-Query Retriever → Contextual Compression → GPT-4o-mini + Structured Output

6.2 Key Features

Feature	Implementation	Purpose
Document Ingestion	PDF, web text, raw strings	Feed research content into the system
Smart Chunking	RecursiveCharacterTextSplitter + timestamps	Preserve context and traceability
Multi-Query Retrieval	MultiQueryRetriever	Better recall from multiple query angles
Contextual Compression	LLMChainExtractor	Only relevant parts returned — less noise
Conversation Memory	Session-based ChatMessageHistory	Follow-up questions work correctly
Structured Output	Pydantic BaseModel + with_structured_output	Typed fields: answer, confidence, sources

6.3 ResearchResponse Schema

```
class ResearchResponse(BaseModel):
    answer: str # Clear answer to the question
    confidence: str # 'high', 'medium', or 'low'
    sources: List[str] # Source documents used
    key_quotes: List[str] # Verbatim quotes from sources
    follow_up_questions: List[str] # Suggested next questions
```

6.4 Multi-Query vs Basic Retrieval

Aspect	Basic Retriever	Multi-Query Retriever
Queries sent to DB	1 (original only)	3-4 (original + LLM-generated variations)
Terminology coverage	Exact words only	Synonyms, rephrasing, related terms
Recall	Lower	Higher
LLM calls	1 (answer only)	2 (query generation + answer)
Best for	Simple keyword queries	Research, academic, complex queries

MODULE 7 — LangGraph: Stateful Agent Workflows

LangGraph is used when a workflow must remember state, route dynamically, loop for self-correction, pause for humans, and resume safely after interruptions. Think of LangChain as building pipelines, and LangGraph as building processes.

7.1 Why LangGraph Exists

Problem with Simple Chains	LangGraph Solution
No looping — can't retry failed steps	Cycles and loops with max-iteration guardrails
No persistent state — context lost between steps	Shared state object travels through all nodes
No conditional routing	Conditional edges route to different nodes based on state
No crash recovery	Checkpointing saves state — resume from any point

7.2 The Three Pillars: State, Nodes, Edges

Pillar	What It Is	Example
State	Shared memory of the graph — everything the workflow needs to remember	messages, step_count, approved, sentiment
Nodes	Functions that read state, do work, and return updates	analyze_sentiment(), generate_response()
Edges	Connections that define where execution goes next	START → analyze → respond → END

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class AgentState(TypedDict):
    input: str
    output: str
    step: int

def process(state: AgentState):
    return {'output': state['input'].upper(), 'step': state['step'] + 1}

graph = StateGraph(AgentState)
graph.add_node('process', process)
graph.add_edge(START, 'process')
graph.add_edge('process', END)
app = graph.compile()
result = app.invoke({'input': 'hello', 'step': 0})
```

7.3 Reducers — How State Gets Merged

When multiple nodes write to the same state field, reducers decide what happens. Without reducers, the last write wins (overwrites). With reducers, you control the merge behaviour.

Reducer	Behaviour	Use When
Default (no annotation)	Last write wins (overwrite)	Only latest value matters: status, score
Annotated[list, operator.add]	Appends to list	Keep all values: findings, drafts, critiques
Annotated[list, add_messages]	Smart merge by message ID	Chat messages — always use this for messages
Custom function	Your own merge logic	Domain-specific: vote counts, confidence scores

7.4 Conditional Edges & Routing

```
from typing import Literal

def route_by_type(state) -> Literal['question', 'command', 'statement']:
    if 'question' in state['query_type']: return 'question'
    elif 'command' in state['query_type']: return 'command'
    return 'statement'

graph.add_conditional_edges('classify', route_by_type,
    {'question': 'handle_question', 'command': 'handle_command', 'statement':
    'handle_statement'})
```

7.5 Loops & Self-Correction

Loops are where LangGraph feels like a real agent system. A self-correcting code generator: generates code → validates it → loops back if incorrect → stops when it succeeds or hits max iterations.

Flow: START → generate → validate → [success? finalize : generate again] → END

📌 *Best practice: Let the LLM generate, but let a deterministic validator judge correctness. This is much stronger than asking the LLM to grade itself.*

7.6 Human-in-the-Loop

Production systems often need a human to review output before execution continues. LangGraph supports this natively with interrupts, state updates, and resume.

```
from langgraph.checkpoint.memory import MemorySaver

memory = MemorySaver()
app = graph.compile(checkpointer=memory, interrupt_before=['approval'])

# Pause before approval node
app.invoke(initial_state, config)

# Human reviews and updates state
app.update_state(config, {'approved': True, 'feedback': 'Looks good'})

# Resume from paused point
app.invoke(None, config)
```

7.7 Checkpointing & Persistence

Without Checkpointing	With Checkpointing
No conversation memory	State survives between invocations
Crashes lose all progress	Resume from saved checkpoints
No clean pause/resume	Supports human review workflows
No branching/forking	Fork from any checkpoint into new threads (A/B testing)

MODULE 8 — LangGraph Multi-Agent Systems

Multi-agent systems divide complex tasks among specialised agents. Each agent has one focused job — smaller scope means better output. LangGraph coordinates them through shared state.

8.1 Tool Calling Agents — The Foundation

A tool calling agent is a LangGraph workflow where the LLM can decide to invoke external tools (functions) before producing a final answer. This is the foundation of agentic behaviour.

```
from langchain_core.tools import tool

@tool
def calculate(expression: str) -> str:
    '''Evaluate a mathematical expression.'''
    try:
        return f'Result: {eval(expression)}'
    except Exception as e:
        return f'Error: {str(e)}'

# Bind tools to LLM
llm_with_tools = llm.bind_tools([calculate, get_weather, search_web])

# ToolNode automatically executes all tool calls
from langgraph.prebuilt import ToolNode
tool_node = ToolNode([calculate, get_weather, search_web])
```

📌 *Golden Rule: Tools should return errors as strings, never raise exceptions. When a tool returns an error string, the LLM reads it, understands what went wrong, and explains it naturally. The agent stays alive and resilient.*

8.2 The ReAct Pattern — Reason, Act, Observe

Phase	What Happens	Message Type
Input	User query arrives	HumanMessage
Reason	LLM decides which tools to call	AIMessage (with tool_calls field)
Act	ToolNode executes the tools	ToolMessage (results)
Observe	LLM reads tool results	AIMessage (no tool_calls)
Final Answer	LLM synthesises and responds	AIMessage (content only)

8.3 Supervisor Pattern — Multi-Agent Orchestration

The supervisor pattern is the most important multi-agent architecture. Think of it as a company org chart: one supervisor (manager) coordinates multiple specialised agents (workers). The supervisor never does the actual work — it only decides who does what next.

Supervisor Responsibility	Description
Understand the task	Parse user intent from the query
Break into subtasks	Decompose complex work into steps
Route to the right agent	Select the appropriate specialist
Collect & synthesise results	Combine agent outputs
Decide when done	Know when to stop and finalise

📌 *Hub and Spoke: No specialist can call another specialist directly. Every handoff goes through the supervisor. This keeps the system predictable and debuggable.*

8.4 Agent Handoffs — Triage & Routing

Pattern	Flow	Best For
Supervisor	Hub & spoke — specialists loop back to supervisor	Iterative, multi-step tasks
Handoff (Triage)	One-way routing — triage → specialist → end	Customer service, routing queries

8.5 Parallel Agent Execution

```
# Fan-OUT: all three agents start simultaneously
graph.add_edge(START, 'research')
graph.add_edge(START, 'creative')
graph.add_edge(START, 'technical')
```

```
# Fan-IN: all three converge at synthesiser
graph.add_edge('research', 'synthesizer')
graph.add_edge('creative', 'synthesizer')
graph.add_edge('technical', 'synthesizer')
```

📌 When multiple edges point FROM the same source node, LangGraph automatically runs those target nodes in parallel. When multiple edges point TO the same node, it waits for all to complete before proceeding.

8.6 Agent Communication Patterns

Pattern	How It Works	Best For
Message Passing	All agents append to one shared conversation thread	Conversational workflows, Q&A chains
Shared State Fields	Each agent gets its own dedicated field in state	Structured pipelines with known outputs
Blackboard Pattern	Multiple agents read/write to shared workspace in a loop until approved	Iterative refinement between two agents

8.7 Hierarchical Multi-Agent Teams

Layer	Role
Layer 1: CEO Supervisor	Top-level router — routes to the right department based on query intent
Layer 2: Department Leads	Research Dept · Content Dept · Analytics Dept (each is a compiled subgraph)
Layer 3: Specialists	Web Researcher · Paper Reviewer · Writer · Editor · Data Analyst

📌 Adding a New Department: Define `build_legal_team()`, compile it, add it as a node. No existing department changes. No existing agent needs to know legal exists. This is the power of hierarchical architecture.

8.8 The Send API — Dynamic Parallel Fan-Out

The Send API solves the problem of not knowing at build time how many parallel agents to spawn. The supervisor decides at runtime (e.g. 2, 3, or 5 searches needed).

```
from langgraph.types import Send

def dispatch_searches(state) -> list[Send]:
    return [
        Send('search_agent', {'search_query': q, 'findings': []})
        for q in state['search_queries']
    ]

graph.add_conditional_edges('supervisor', dispatch_searches, ['search_agent'])
```


MODULE 9 — Production Deployment

Building an agent is only half the job. Deploying it safely, reliably, and cost-effectively is the other half. This module covers observability, security, testing, error handling, cost optimisation, and deployment.

9.1 Observability — The Three Pillars

Pillar	Answers	Key Metrics
Traces	What happened?	Agent flow, inputs/outputs, tool calls, decisions made
Metrics	How much / how fast?	Token count, latency per node, cost per run, error rates
Evals	Is it good?	Correctness, relevance, human feedback, regression detection

```
# LangSmith Setup — just 2 environment variables
LANGCHAIN_TRACING_V2=true
LANGCHAIN_API_KEY=your_key_here
LANGCHAIN_PROJECT=my-project-name

# Make any function traceable
from langsmith import traceable

@traceable(name='my_chain', tags=['production'])
def my_chain(query: str) -> str:
    return llm.invoke(query).content
```

9.2 Security — 5-Layer Architecture

Layer	What It Does	Cost
1. Input Sanitiser	Regex blocks known injection patterns (ignore previous instructions, etc.)	Free — instant
2. PII Detector	Masks email, phone, SSN, credit card before LLM sees it	Free — regex
3. LLM Guard	Secondary LLM classifies input as safe/unsafe	API cost — run last
4. LLM Processing	The actual work — input is clean and secure by this point	Normal cost
5. Output Validator	Catches PII leaks and harmful content on the way out	Free — regex

❑ *Fail fast, fail cheap: Run regex first (free). Only call LLM Guard if regex passes. This avoids unnecessary API calls for obvious attacks.*

9.3 Testing Strategy — The Pyramid

Level	Type	When to Run	Cost
Base	Unit tests with mocks	Every commit	Free — no API calls
Middle	Integration tests (real LLM)	Before every deploy	Low — real API calls
Upper	Regression tests (LLM-as-judge)	Nightly	Medium
Top	LangSmith eval datasets	Weekly / pre-release	Higher

❑ *Key insight: Test your code, not the LLM's intelligence. Unit tests with mocks are free, fast, and deterministic. For integration tests, assert CONTAINS keywords — not exact match — because LLM output is non-deterministic.*

9.4 Production Error Handling

Pattern	When to Use	How It Works
---------	-------------	--------------

Retry with Exponential Backoff	API timeouts, transient failures	Retry 3× with 1s, 2s, 4s waits
Circuit Breaker	Repeated failures (dead server)	After N failures: stop retrying for 30s, then probe
LLM Fallback Chain	Primary model fails	GPT-4o → Claude → GPT-4o-mini → graceful error
Graph-Based Error Handling	Complex agents needing visibility	LangGraph state machine routes failures gracefully

9.5 Cost Optimisation

Strategy	How	Savings
Model Routing	Classify query complexity; use cheap model for simple tasks	60-80% on simple queries
Semantic Caching	Cache responses by query hash; serve from cache on repeat	30-50% typical hit rate
Token Budgeting	Reject oversized inputs before LLM call	Prevents runaway costs

📌 *Output tokens cost 4-5× more than input tokens. Optimise output length aggressively with `max_tokens`. Combine model routing + caching + token budgeting for 50-80% cost reduction.*

9.6 Production API Architecture

Complete request pipeline — every incoming request travels through all layers:

Layer	Module	What It Does
1. Rate Limiter	SlowAPI	Throttle by IP — 20 req/min default; returns HTTP 429 if exceeded
2. Security Middleware	security.py	Injection check + PII masking on input
3. Cache Lookup	cache.py	Return cached response if hit (free + near-zero latency)
4. LangGraph Agent	agent.py	Primary model → retry → fallback model
5. Output Validator	security.py	PII masking + harmful content check on output
6. Cache Store	cache.py	Cache successful response for future hits
7. Metrics + Logging	monitoring.py	Record latency, tokens, errors, cache stats
8. JSON Response	models.py	Structured ChatResponse with all metadata

9.7 Docker & Cloud Deployment

Principle	Implementation	Why It Matters
Layer Caching	Copy <code>pyproject.toml</code> before app code	If only code changes, dependency install is cached — faster rebuilds
Non-Root User	<code>useradd appuser + USER appuser</code>	Security best practice — container compromise can't escalate to host root
Health Check	<code>HEALTHCHECK</code> hits <code>/health</code> every 30s	Docker marks container unhealthy after 3 failures — orchestrators restart it

Deploy to Render with `render.yaml` — infrastructure as code. Every git push to main automatically rebuilds and redeloys your API. No SSH, no manual restarts.

⚡ QUICK REFERENCE CHEAT SHEETS

LCEL Pattern Reference

Task	Code Pattern
Basic chain	<code>chain = prompt model parser</code>
Single invoke	<code>chain.invoke({'key': 'value'})</code>
Batch	<code>chain.batch([{'key': 'v1'}, {'key': 'v2'}])</code>
Stream	<code>for chunk in chain.stream({'key': 'v'}): print(chunk, end="")</code>
Structured output	<code>model.with_structured_output(MySchema)</code>
Universal model init	<code>init_chat_model('gpt-4o-mini', model_provider='openai')</code>
LangSmith trace	<code>@traceable(name='my_func', run_type='chain')</code>
Parallel chains	<code>RunnableParallel(a=chain1, b=chain2)</code>
Conditional routing	<code>RunnableBranch((condition_fn, chain1), default_chain)</code>

RAG Strategy Selector

Problem	Strategy	Key Class
Missing relevant docs	Multi-Query Retriever	MultiQueryRetriever
Too much noise in context	Contextual Compression	ContextualCompressionRetriever
Missing exact keyword matches	Hybrid Search	EnsembleRetriever + BM25Retriever
Precision vs context tradeoff	Parent Doc Retriever	ParentDocumentRetriever
Need typed/structured output	Structured RAG	<code>llm.with_structured_output()</code>
Out-of-scope questions	Fallback Prompt	Prompt engineering

Memory Strategy Selector

Use Case	Strategy	Key Class
Short demo / testing	Full Buffer (in-memory)	ChatMessageHistory
Long conversations, cost control	Sliding Window	Custom WindowChatHistory
Long conversations, fact retention	Summary Memory	Custom + StrOutputParser
Production chatbot	SQLite Persistence	SQLChatMessageHistory
Multi-user system	Session-based store	RunnableWithMessageHistory
Context window overflow	Message Trimming	<code>trim_messages()</code>

Multi-Agent Pattern Selector

Pattern	Mental Model	Use When
Tool Calling Agent	One brain, many hands	Single agent needs external tools
Supervisor	Project manager + team	Complex multi-step tasks, iterative work
Handoff / Triage	Receptionist + departments	Routing queries to specialists
Parallel Execution	Multiple workers, same deadline	Independent tasks that can run simultaneously
Map-Reduce	Divide, process, combine	Processing N documents/items
Blackboard	Shared whiteboard in a war room	Iterative refinement between two agents
Hierarchical	Company org chart	Large systems with many specialised agents

Reducer Quick Reference

Need	Reducer
------	---------

Only latest value matters	Default (no annotation)
Keep all values in a list	Annotated[list, operator.add]
Chat messages with deduplication	Annotated[list, add_messages]
Custom merge logic	Annotated[dict, your_function]

Production Checklist

Category	Checklist Item
Security	Input sanitisation — blocks prompt injection
Security	PII detection and masking on both input AND output
Security	Rate limiting — throttles by IP
Security	Secrets in environment variables — never hard-coded
Reliability	Retry with exponential backoff
Reliability	Circuit breaker — stops retrying after N failures
Reliability	Fallback chain — primary → secondary → graceful error
Reliability	Health check endpoint /health
Performance	Response caching with TTL
Performance	Model routing — cheap model for simple tasks
Performance	Token budgeting — reject oversized inputs
Observability	LangSmith tracing on every request
Observability	Structured JSON logging
Observability	Metrics collection — latency, tokens, errors, cache hit rate
Deployment	Dockerfile with non-root user, layer caching, health check
Deployment	render.yaml — infrastructure as code
Deployment	Auto-deploy on git push to main
Deployment	20+ unit tests written and passing

🔑 KEY MENTAL MODELS TO REMEMBER

Concept	Mental Model
LangChain vs LangGraph	LangChain = pipelines (straight lines). LangGraph = processes (graphs with loops, branches, memory).
LCEL	Everything is a Runnable. Pipe them together with . Compose freely.
RAG	Index once. Retrieve per query. Ground the LLM in real documents.
Memory	Session ID = isolated memory. MessagesPlaceholder = where history is injected.
Reducers	Default = overwrite. operator.add = accumulate. add_messages = smart merge for chat.
Supervisor	Hub and spoke. Specialists never talk to each other directly.
Tools	Return errors as strings, never raise exceptions. The LLM handles it conversationally.
Production	Security layers → Cache → Agent → Output validation → Metrics. In that order.
Cost	Output tokens cost 4-5× more than input. Route cheap, cache aggressively, budget tokens.
Testing	Test your code (mocks, free). Test your LLM (integration, expensive). Run in that order.

🔑 *You now have the complete Agentic AI curriculum in one place. Start with Module 1, build each concept, and by Module 9 you will have the skills to build and deploy production-grade AI agents.*